# NumPy?

NumPy (Numerical Python) is a Python library used for numerical computing. It provides support for large, multi-dimensional arrays and matrices, along with many mathematical functions to operate on these arrays efficiently.

Main purpose: **Fast mathematical and scientific computation in Python**

## NumPy is widely used in:

- Data Science
- Machine Learning
- Scientific Computing
- Image Processing
- Artificial Intelligence

Many libraries such as **TensorFlow, Pandas, and SciPy** depend on **NumPy.**

## Why Do We Need NumPy?

Normal Python **lists** have limitations when working with large numerical data.

**Problems with Python Lists**

1. Slow for large calculations
2. Cannot perform vectorized mathematical operations easily
3. Consume more memory
4. Hard to perform matrix operations

## NumPy Solves These Problems

NumPy provides:

| **Feature** | **Benefit** |
|---|---|
| Fast array operations | Uses optimized C code |
| Vectorization | No need for loops |
| Memory efficient | Uses less memory |
| Mathematical functions | Built-in opertaions |

| Matrix operations | Easy linear algebra |
|---|---|

**Example Comparison**

Python list addition:

```
a = [1,2,3]
b = [4,5,6]

result = []
for i in range(len(a)):
    result.append(a[i] + b[i])

print(result)
```

**NumPy version:**

```
import numpy as np

a = np.array([1,2,3])
b = np.array([4,5,6])

print(a + b)
```

Output= [5 7 9]

NumPy performs the operation **much faster**.

**Installing NumPy**

Install using pip: **pip install numpy**

Import NumPy in Python: **import numpy as np**

**np** is commonly used alias.

## NumPy Array

The **core object of NumPy is the ndarray (N-dimensional array).**

It is a **grid of values of the same type indexed by integers**.

Example:

```
import numpy as np

arr = np.array([1,2,3,4])

print(arr)
```

**Output=[1 2 3 4]**

## Types of NumPy Arrays

### 1D Array

```
arr = np.array([10,20,30])

print(arr)
```

Out=[10 20 30]

### 2D Array

```
arr = np.array([[1,2,3],[4,5,6]])

print(arr)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

### 3D Array

```
arr = np.array([[[1,2],[3,4]],[[5,6],[7,8]]])
print(arr)
```

## Creating Special Arrays

### Zeros Array

```
np.zeros((3,3))
```

Output:

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

### Ones Array

```
np.ones((2,3))
```

Out:

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

### Range Array

```
np.arange(0,10)
```

Out:[0 1 2 3 4 5 6 7 8 9]

### Identity Matrix

```
np.eye(3)
```

Out:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

## NumPy Array Operations

### Addition

```
a = np.array([1,2,3])
b = np.array([4,5,6])

print(a + b)
```

Out: [5 7 9]

### Multiplication

```
print(a * b)
```

Out:[4 10 18]

### Scalar Operations

```
a = np.array([1,2,3])

print(a * 2)
```

Out:[2 4 6]

## Statistical and Aggregation Functions

### Square Root

```
np.sqrt([4,9,16])
```

Out:[2. 3. 4.]

### Sum

```
a = np.array([1,2,3,4])

np.sum(a)
```

Out:10

**Mean and Maximum**

np.mean(a)  np.max(a)

Out:2.5  Out:4

**Indexing and Slicing**

**Indexing :** accessing single element from array

```
arr = np.array([10,20,30,40])

print(arr[1])
```

Out:20

**2D Array Indexing**

```
arr = np.array([[1,2,3],[4,5,6]])

print(arr[1,2])  #it takes the first row and 2nd column
```

Here arr=[[1 2 3]  so index start from 0  in row wise 0 and 1 and in column wie 0,1,2
[4 5 6]]

So arr[1,2] means 1st index row and 2nd index column so common element 6
row 1 column 2

Out:6

**Slicing:**Getting multiple elements

```
arr = np.array([10,20,30,40,50])

print(arr[1:4])
```

Out:[20 30 40]

## Reshaping Arrays

**Reshaping** means **changing the shape (dimensions) of an array without changing its data**.

In NumPy, we use the **reshape()** function to convert a 1-dimensional array into a **2D or multi-dimensional array**.

**Why Reshaping is Needed**

Sometimes data is stored in a **single row (1D)** but we need it in **rows and columns (2D)** for:

- Matrix calculations
- Machine learning models
- Image processing
- Data analysis

Reshaping helps organize data properly.

```
arr = np.arange(6)        #created a 1D array of 6 elements [0 1 2 3 4 5]

print(arr.reshape(2,3))   #Now the array is converted into a 2 rows × 3 columns matrix.
```

Out:

```
[[0 1 2]
 [3 4 5]]
```

## Flatten Example

**Convert multi-dimensional array into 1D.**

```
arr = np.array([[1,2],[3,4]])

print(arr.flatten())
```

Out:[1 2 3 4]

## Arithmetic, Logarithmic, and Trigonometric Functions

### Arithmetic Operations

```
a = np.array([1,2,3])
b = np.array([4,5,6])

print(a + b)
print(a * b)
```

**Out:**

```
[5 7 9]
[ 4 10 18]
```

### Logarithmic Function

```
np.log([1,2,3])
```

Out: [0.  0.693  1.098]

### Trigonometric Function

```
np.sin([0, np.pi/2])
```

Out: [0. 1.]

### Create array using arange()

```
arr = np.arange(1,10)

print(arr)
```

Out:[1 2 3 4 5 6 7 8 9]

## Rounding, Comparison, and Logical Operations

### Rounding

```
arr = np.array([1.2, 2.7, 3.5])

print(np.round(arr))
```

Out:[1. 3. 4.]

### Comparison

```
arr = np.array([10,20,30])

print(arr > 15)
```

Out: [False True True]

### Logical Operations

```
a = np.array([True, False, True])
b = np.array([False, False, True])

print(np.logical_and(a,b))
```

Out: [False False True]

### Broadcasting Rules

Broadcasting allows NumPy to **perform operations on arrays of different sizes**.

```
a = np.array([1,2,3])
b = 2
```

```
print(a + b)
```

Out: [3 4 5] here 2 is added to every element

## Linear Algebra

NumPy provides functions for **matrix operations**.

**Matrix Multiplication**

```
a = np.array([[1,2],[3,4]])
b = np.array([[5,6],[7,8]])

print(np.dot(a,b))
```

**Out:**

```
[[19 22]
 [43 50]]
```

**Determinant :** np.linalg.det(a)

**Transpose:** a.T  a is ([[1,2],[3,4]])  exchange the rows to columns

Out:

```
[[1 3]
[2 4]]
```

## Random Sampling

Used to **generate random numbers**.

**Random number**

np.random.rand()

Out: 0.537

**Random array**

np.random.rand(2,3)

Out:
[[0.21 0.45 0.67]
 [0.11 0.90 0.32]]

**Random integers**

np.random.randint(1,10,5)

Out:[3 7 1 9 5]

**Advantages of NumPy**

1. High performance
2. Memory efficient
3. Supports large datasets
4. Provides many mathematical functions
5. Supports multi-dimensional arrays
6. Used in machine learning and data science

# Matplotlib

Matplotlib is a Python library used for **data visualization**. It helps create graphs, charts, and plots to represent data visually.

Using **Matplotlib**, we can create:

- Line graphs
- Bar charts
- Pie charts
- Histograms
- Scatter plots
- Subplots
- 3D plots

It is commonly used with **NumPy** and **Pandas** for data analysis and visualization.

## Why Do We Need Matplotlib?

Data in numbers is often hard to understand. Visualization helps us **see patterns, trends, and comparisons easily**.

Example:

Instead of reading numbers: 10, 20, 30, 40, 50

A graph can show:

- Growth
- Decline
- Relationship between variables

Matplotlib helps convert **numerical data into visual graphs**.

## Installing Matplotlib

Install using pip: **pip install matplotlib**

Import in Python: **import matplotlib.pyplot as plt**

**`pyplot` is the most commonly used module.**

## Basic Structure of a Plot

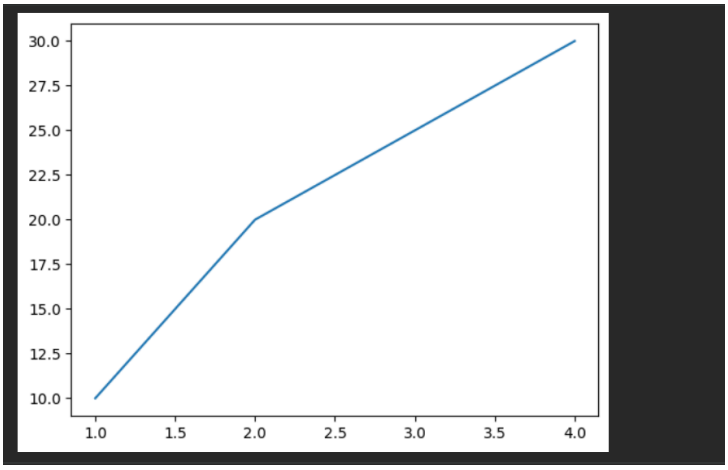
**Pyplot** is basically used for 1d ploting

**Most plots follow this structure:**

```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y = [10,20,25,30]

plt.plot(x,y)
plt.show()
```

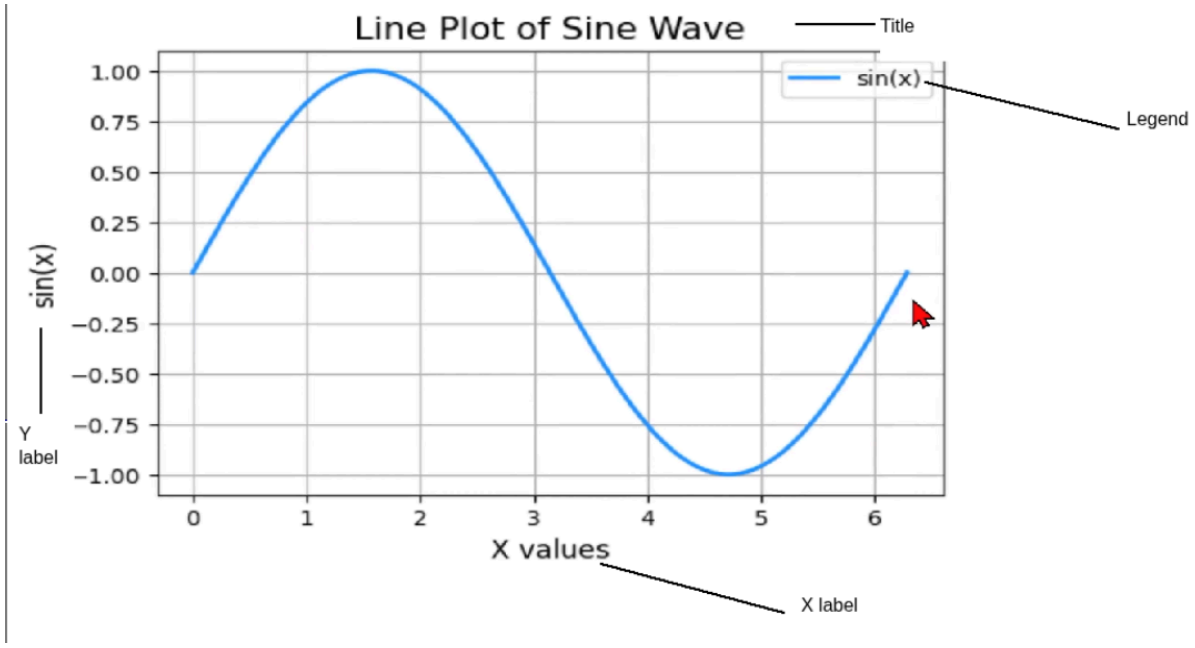
Explanation:

- `x` → values on x-axis
- `y` → values on y-axis
- `plt.plot()` → creates graph
- `plt.show()` → displays graph




## Can add like

```
plt.figure(figsize=(6,4))  #create a new figure with custom size
plt.plot(x,y, label='sin(x)' , color='blue' , linestyle='-' , linewidth=2)
plt.title("Line plot of sine wave", fontsize=15)
plt.xlabel("X values", fontsize=12)
plt.ylabel("sin(x) , fontsize=12)
plt.legend()   #Show a legend
plt.grid(True) #Add grid lines for better readability
plt.show()    #Display the plot
```

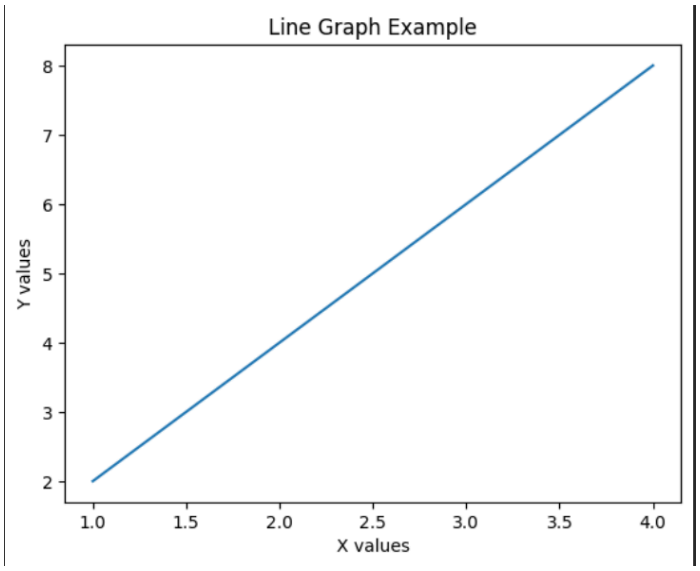



Grid means the vertical and horizontal lines

## Line Plot

A **line plot** shows how data changes over time.

```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y = [2,4,6,8]

plt.plot(x,y)
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Line Graph Example")

plt.show()
```

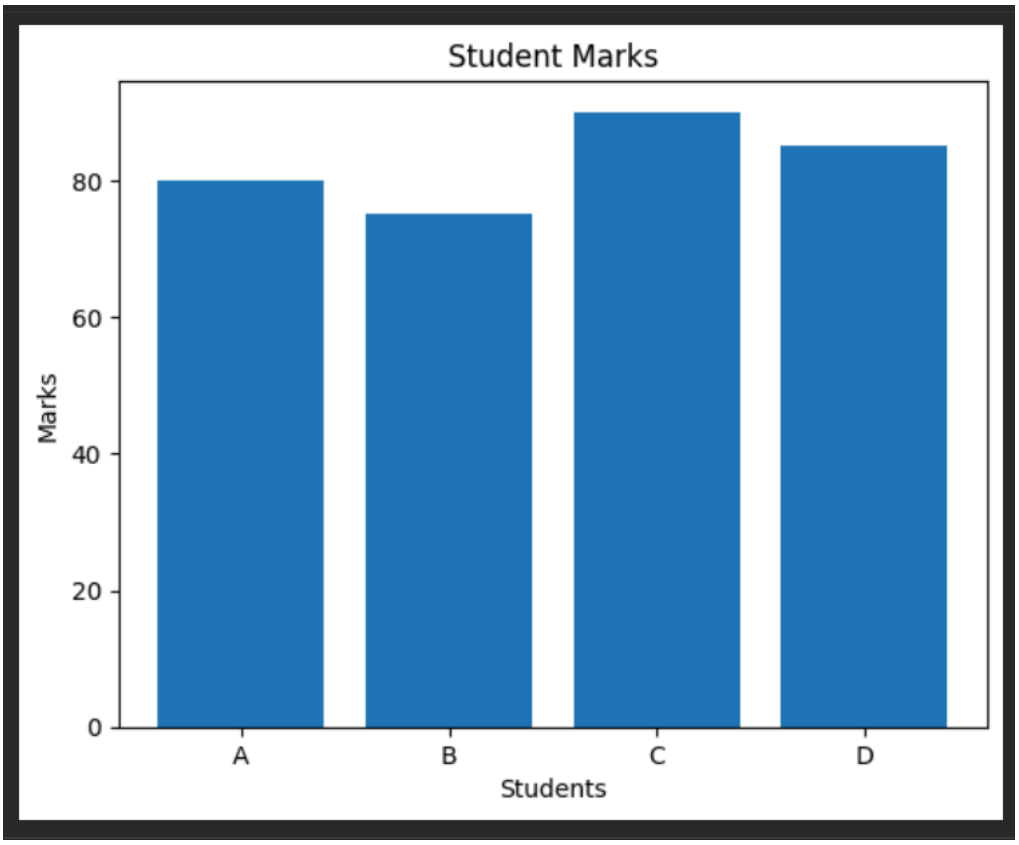


## Bar Chart

Bar charts are used to **compare categories**.

Example: comparing student marks.

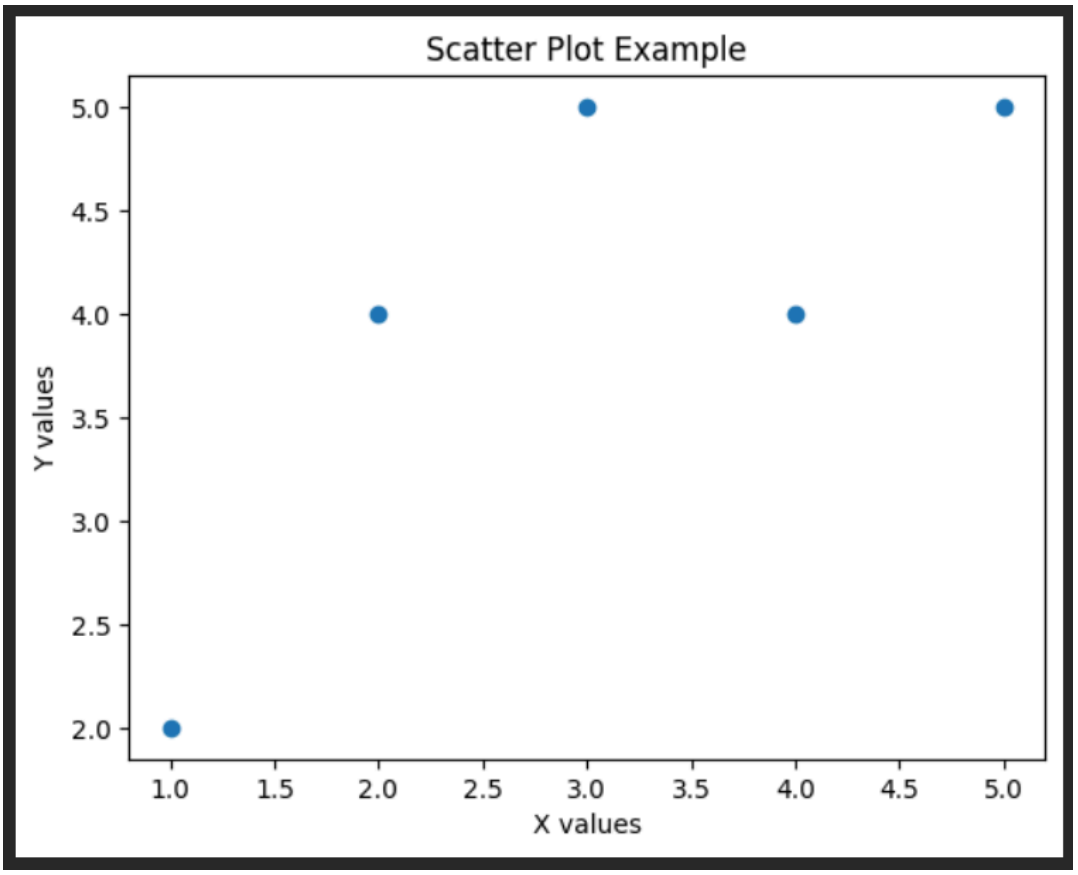
```
import matplotlib.pyplot as plt

students = ["A","B","C","D"]
marks = [80,75,90,85]

plt.bar(students, marks)
plt.title("Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")

plt.show()
```



## Scatter Plot

Scatter plots show the **relationship between two variables**.

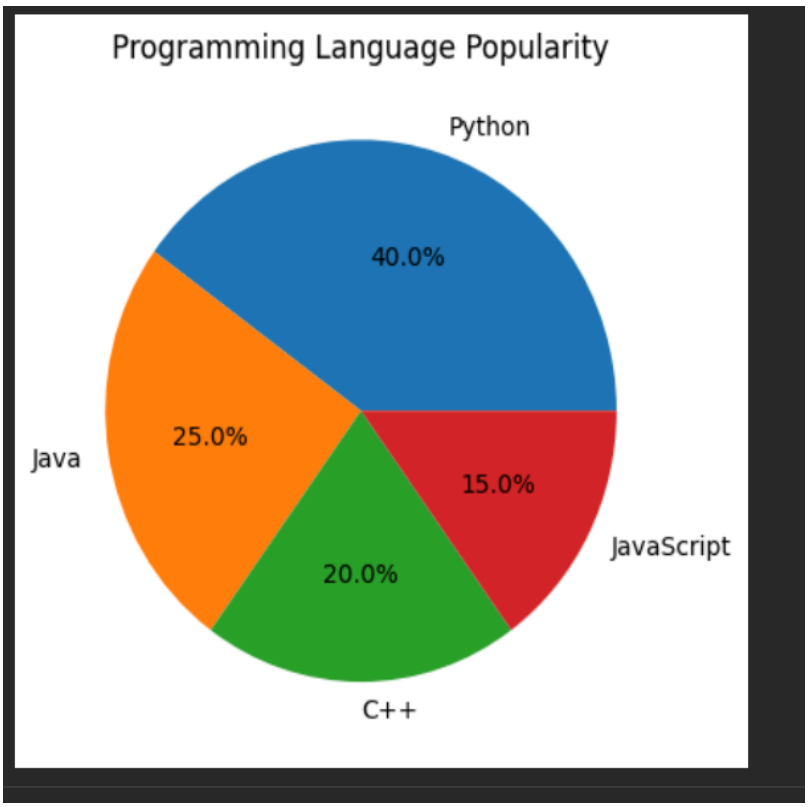
```
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [2,4,5,4,5]

plt.scatter(x,y) =>can pass(x,y, c='red' , marker='o' ,alpha=1)  such values it change the looks of plot

plt.title("Scatter Plot Example")
plt.xlabel("X values")
plt.ylabel("Y values")

plt.show()
```



## Pie Chart

Pie charts represent **percentage distribution**.

```
import matplotlib.pyplot as plt

labels = ["Python","Java","C++","JavaScript"]
sizes = [40,25,20,15]

plt.pie(sizes, labels=labels, autopct='%1.1f%%')

plt.title("Programming Language Popularity")

plt.show()
```

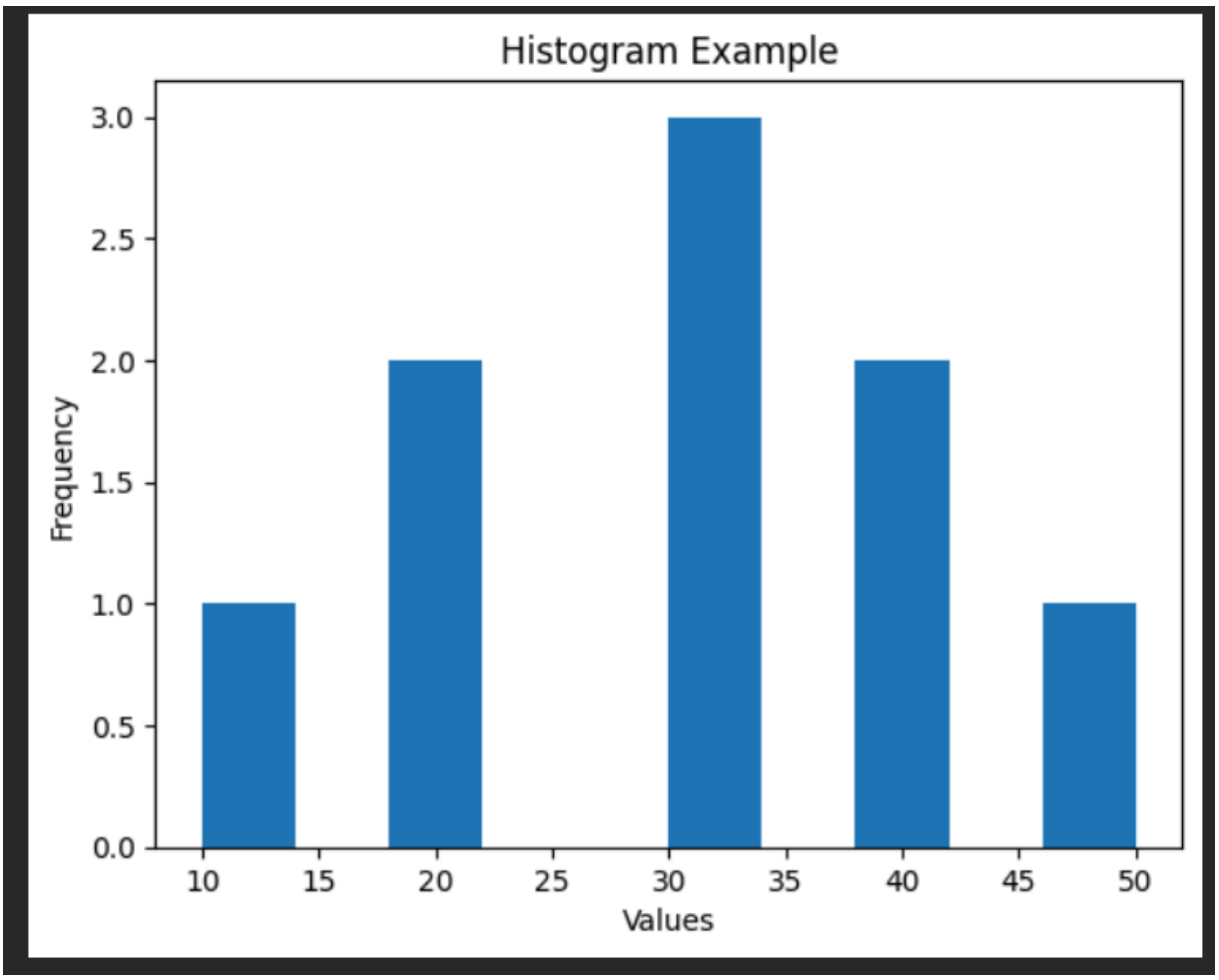


## Histogram

Histogram shows **distribution of numerical** data.

it is continues

```
import matplotlib.pyplot as plt

data = [10,20,20,30,30,30,40,40,50]

plt.hist(data)

plt.title("Histogram Example")
plt.xlabel("Values")
plt.ylabel("Frequency")

plt.show()
```



## Box Plot

A **box plot** (also called a **box-and-whisker plot**) is a graph used to show the **distribution of numerical data based on five summary statistics**. It helps you understand how the data is spread and whether there are any **outliers**.

In **Matplotlib**, a box plot is created using `plt.boxplot()`.

### What a Box Plot Shows

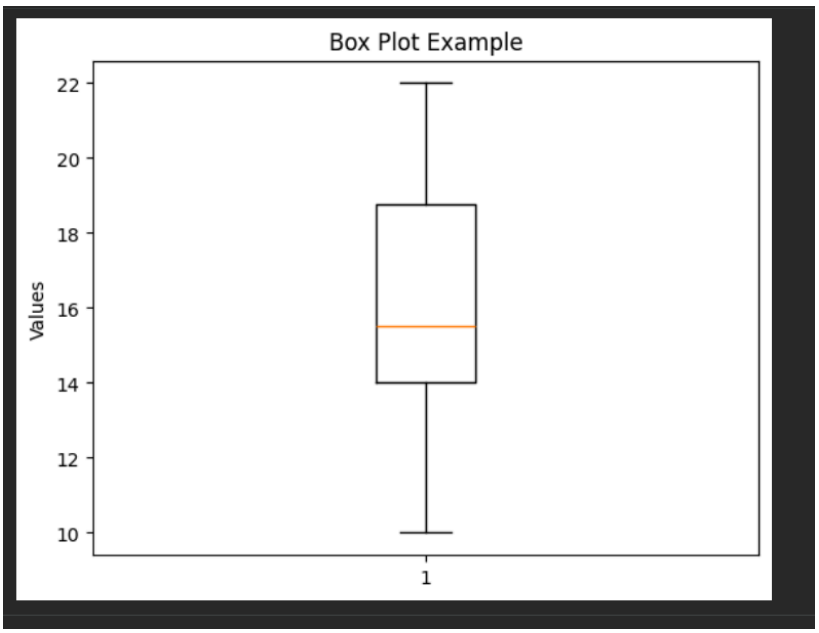
A box plot summarizes data using five important values:

1. **Minimum** – the smallest value in the dataset (excluding outliers).
2. **First Quartile (Q1)** – the value below which 25% of the data falls.
3. **Median (Q2)** – the middle value of the dataset.
4. **Third Quartile (Q3)** – the value below which 75% of the data falls.
5. **Maximum** – the largest value in the dataset (excluding outliers).

### Parts of the Box Plot

- The **box** represents the range from **Q1 to Q3** (called the interquartile range).
- The **line inside the box** shows the **median**.
- The **whiskers** extend from the box to the minimum and maximum values.
- **Points outside the whiskers** are considered **outliers**.

```
import matplotlib.pyplot as plt

data = [12, 15, 14, 10, 18, 20, 22, 19, 16, 14]

plt.boxplot(data)

plt.title("Box Plot Example")

plt.ylabel("Values")

plt.show()
```




## SubPlot

a **subplot** is used to display **multiple plots in a single figure**. This helps compare different graphs side-by-side or in a grid layout.

The function used is **`plt.subplot()`**.

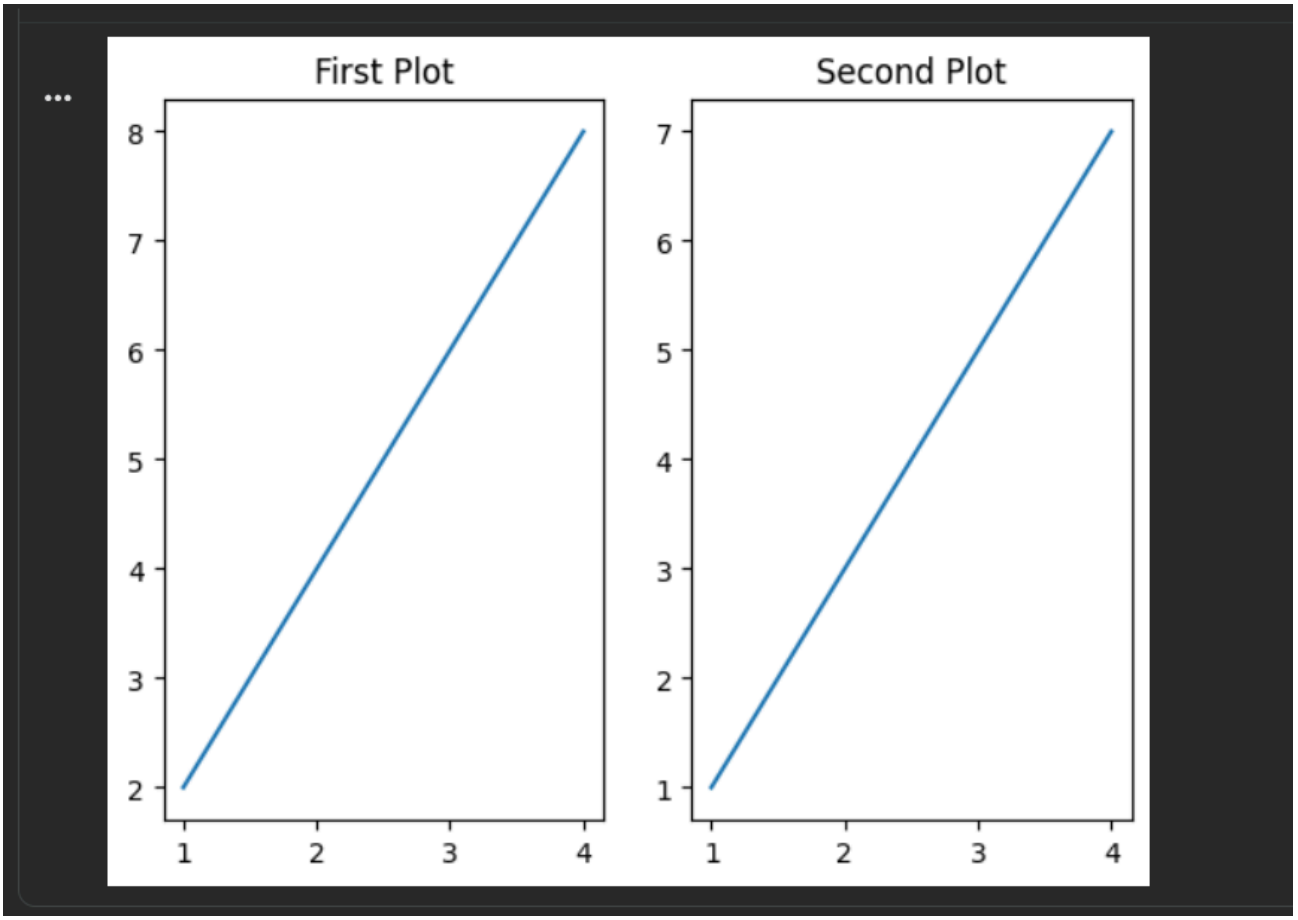
### How it Works

```
plt.subplot(rows, columns, index)
```

- **rows** → number of rows in the figure
- **columns** → number of columns in the figure
- **index** → position of the current plot (starts from 1)

**Example**

```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y1 = [2,4,6,8]
y2 = [1,3,5,7]

plt.subplot(1,2,1)   # 1 row, 2 columns, position 1
plt.plot(x,y1)
plt.title("First Plot")

plt.subplot(1,2,2)   # 1 row, 2 columns, position 2
plt.plot(x,y2)
plt.title("Second Plot")

plt.show()
```



In **Matplotlib**, a **bar chart** and a **histogram** may look similar because both use rectangular bars, but they represent different kinds of information.

## Bar Chart

A **bar chart** is used to compare values between **different categories**. Each bar represents a separate category, such as products, cities, or students. The bars usually have **spaces between them** to show that the categories are independent of each other.

In Matplotlib, a bar chart is created using `plt.bar()`.

Example:
If you want to compare the sales of Product A, Product B, and Product C, you would use a bar chart.

## Histogram

A **histogram** is used to show the **distribution of continuous numerical data**. Instead of categories, the x-axis represents **ranges of values (called bins)**. The bars usually **touch each other** because the ranges are continuous.

In Matplotlib, a histogram is created using `plt.hist()`.

Example:
If you want to see how exam scores are distributed (like how many students scored between 0–10, 10–20, etc.), you would use a histogram.

### Key Idea

A **bar chart compares categories**, while a **histogram shows how numerical data is distributed across value ranges**.

# Pandas

pandas is an open-source Python library used for data analysis and data manipulation. It helps programmers easily read, organize, clean, and analyze data, especially data in table form.

Pandas is widely used in data science, machine learning, and data analysis because it provides powerful tools to work with structured data.

**Main Data Structures in Pandas**

**1. Series**

A Series is a one-dimensional labeled array that can store data like numbers, strings, or objects.
It is similar to a single column in a table.

Example:

```
import pandas as pd

data = pd.Series([10, 20, 30, 40])
print(data)
```

**Out:**

```
0    10
1    20
2    30
3    40
dtype: int64
```

**2. DataFrame**

A **DataFrame** is a **two-dimensional table-like structure** with rows and columns.
It is similar to a **spreadsheet or database table**.

Example:

```
import pandas as pd

data = {
    "Name": ["Ali", "Sara", "John"],
    "Age": [20, 21, 19]
}

df = pd.DataFrame(data)
print(df)
```

**Out:**

```
   Name  Age
0   Ali   20
1  Sara   21
2  John   19
```

**Uses of Pandas**

Pandas is used to:

- **Read data** from files such as CSV, Excel, and databases
- **Clean data** by removing or filling missing values
- **Filter and sort data**
- **Analyze data** and perform calculations
- **Prepare data for visualization or machine learning**

**Example: Reading a CSV File**

```
import pandas as pd

df = pd.read_csv("data.csv")
print(df.head())
```

Here, Pandas reads the CSV file and displays the first few rows of the dataset.

✅ **In simple terms:**
Pandas helps you **work with large datasets easily by organizing them into tables and providing tools to analyze them efficiently.**

```
import pandas as pd
import numpy as np
import random


random.seed(101)

names=['Aarav Patel','Sophia Smith','Liam Johnson','Saanvi Gupta','Olivia Brown','Indiana Jones','Rohan Sharma','James Wilson']


genders=['Male','Female','Male','Female','Female','Male','Male','Male']


ages=[random.randint(21,25) for _ in range(8)]
marks=[random.randint(60,95) for _ in range(8)]

subjects=random.choices(['Python','Statistics','Generative AI'],k=8)


df=pd.DataFrame({
    'Name':names,
    'Gender':genders,
    'Age':ages,
    'Marks':marks,
    'Subject':subjects
})

Df
```

This code uses **pandas**, **NumPy**, and Python's **random** module to **create a sample dataset of students and store it in a Pandas DataFrame**.

**random.seed(101)**

This sets the **starting point for the random number generator**.

Normally, random values change every time the code runs.
By setting a **seed value (101)**, the random numbers generated will be **the same every time you run the code**.

**Example:**
If the seed is 101, the generated ages and marks will always be the same sequence.

Why it is used:

- To make results **reproducible**
- Useful in **data science experiments and testing**

| | Name | Gender | Age | Marks | Subject |
|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 74 | Python |
| 1 | Sophia Smith | Female | 22 | 78 | Python |
| 2 | Liam Johnson | Male | 25 | 91 | Generative AI |
| 3 | Saanvi Gupta | Female | 23 | 73 | Statistics |
| 4 | Olivia Brown | Female | 24 | 81 | Statistics |
| 5 | Indiana Jones | Male | 21 | 88 | Python |
| 6 | Rohan Sharma | Male | 25 | 64 | Statistics |
| 7 | James Wilson | Male | 22 | 76 | Statistics |

df['Marks']

| | Marks |
|---|---|
| 0 | 74 |
| 1 | 78 |
| 2 | 91 |
| 3 | 73 |
| 4 | 81 |
| 5 | 88 |
| 6 | 64 |
| 7 | 76 |

df[df['Marks']>80]

| | Name | Gender | Age | Marks | Subject |
|---|---|---|---|---|---|
| 2 | Liam Johnson | Male | 25 | 91 | Generative AI |
| 4 | Olivia Brown | Female | 24 | 81 | Statistics |
| 5 | Indiana Jones | Male | 21 | 88 | Python |

df[['Name','Marks']]

| | Name | Marks |
|---|---|---|
| 0 | Aarav Patel | 74 |
| 1 | Sophia Smith | 78 |
| 2 | Liam Johnson | 91 |
| 3 | Saanvi Gupta | 73 |
| 4 | Olivia Brown | 81 |
| 5 | Indiana Jones | 88 |
| 6 | Rohan Sharma | 64 |
| 7 | James Wilson | 76 |

In **pandas**, `loc` and `iloc` are used to **select or access data from a DataFrame**, but they work in different ways.

### `loc`

`loc` is used to select data **by labels (row names and column names)**.

- It uses **actual row labels and column labels**.
- Works with **names, conditions, or label ranges**.

Example:

```
df.loc[0, 'Name']
```

Explanation:
This selects the **value in row label 0 and column "Name"**.

Another example:

```
df.loc[0:3, ['Name','Marks']]
```

This selects:

- rows **0 to 3**
- columns **Name and Marks**

## `iloc`

`iloc` is used to select data **by integer position (index number)**.

- It uses **numerical index positions**
- Similar to how lists work in Python.

Example:

```
df.iloc[0, 0]
```

Explanation:
This selects the **first row and first column**.

Another example:

```
df.iloc[0:3, 1:3]
```

This selects:

- rows **0 to 2**
- columns **1 to 2**

**Main Difference**

- **`loc` → label-based indexing (names of rows and columns)**
- **`iloc` → position-based indexing (index numbers)**

```
df['Double_Marks']=df['Marks']*2
```

| | Name | Gender | Age | Marks | Subject | Double_Marks |
|---|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 74 | Python | 148 |
| 1 | Sophia Smith | Female | 22 | 78 | Python | 156 |
| 2 | Liam Johnson | Male | 25 | 91 | Generative AI | 182 |
| 3 | Saanvi Gupta | Female | 23 | 73 | Statistics | 146 |
| 4 | Olivia Brown | Female | 24 | 81 | Statistics | 162 |
| 5 | Indiana Jones | Male | 21 | 88 | Python | 176 |
| 6 | Rohan Sharma | Male | 25 | 64 | Statistics | 128 |
| 7 | James Wilson | Male | 22 | 76 | Statistics | 152 |

```
df['Marks']=df['Marks'].apply(lambda x:x+5)
df
```

| | Name | Gender | Age | Marks | Subject | Double_Marks |
|---|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 79 | Python | 148 |
| 1 | Sophia Smith | Female | 22 | 83 | Python | 156 |
| 2 | Liam Johnson | Male | 25 | 96 | Generative AI | 182 |
| 3 | Saanvi Gupta | Female | 23 | 78 | Statistics | 146 |
| 4 | Olivia Brown | Female | 24 | 86 | Statistics | 162 |
| 5 | Indiana Jones | Male | 21 | 93 | Python | 176 |
| 6 | Rohan Sharma | Male | 25 | 69 | Statistics | 128 |
| 7 | James Wilson | Male | 22 | 81 | Statistics | 152 |

```
df=df.rename(columns={'Marks':'Updated_Marks'})
```

| | Name | Gender | Age | Updated_Marks | Subject | Double_Marks |
|---|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 79 | Python | 148 |
| 1 | Sophia Smith | Female | 22 | 83 | Python | 156 |
| 2 | Liam Johnson | Male | 25 | 96 | Generative AI | 182 |
| 3 | Saanvi Gupta | Female | 23 | 78 | Statistics | 146 |
| 4 | Olivia Brown | Female | 24 | 86 | Statistics | 162 |
| 5 | Indiana Jones | Male | 21 | 93 | Python | 176 |
| 6 | Rohan Sharma | Male | 25 | 69 | Statistics | 128 |
| 7 | James Wilson | Male | 22 | 81 | Statistics | 152 |

```
df.drop(['Double_Marks'], axis=1 ,inplace=True)
df
```

| | Name | Gender | Age | Updated_Marks | Subject |
|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 79 | Python |
| 1 | Sophia Smith | Female | 22 | 83 | Python |
| 2 | Liam Johnson | Male | 25 | 96 | Generative AI |
| 3 | Saanvi Gupta | Female | 23 | 78 | Statistics |
| 4 | Olivia Brown | Female | 24 | 86 | Statistics |
| 5 | Indiana Jones | Male | 21 | 93 | Python |
| 6 | Rohan Sharma | Male | 25 | 69 | Statistics |
| 7 | James Wilson | Male | 22 | 81 | Statistics |

is used to **remove the column `Double_Marks` from the DataFrame**.
The parameters **`axis`** and **`inplace`** control how the drop operation works.

**axis**

`axis` tells Pandas **whether you want to remove a row or a column**.

- `axis=0` → operate on **rows** (default)
- `axis=1` → operate on **columns**

In code axis=1

means **drop a column**.

So Pandas removes the **`Double_Marks` column** from the DataFrame.

**inplace**

`inplace` tells Pandas **whether to modify the original DataFrame or create a new one**.

- `inplace=True` → changes the **original DataFrame directly**
- `inplace=False` → creates a **new DataFrame with the change**

In Code inplace=True  means the column is removed **permanently from `df` itself**.

| index | Name | Marks |
|---|---|---|
| 0 | Aarav | 85 |
| 1 | Sophia | 90 |
| 2 | Liam | 78 |

Droping a Row Using axis=0

df.drop(1, axis=0, inplace=True)

Explanation:

- `1` → row index
- `axis=0` → row operation
- `inplace=True` → modify original DataFrame

After dropping row **1**:

| index | Name | Marks |
|---|---|---|
| 0 | Aarav | 85 |
| 2 | Liam | 78 |

After dropping row **1**:

```
df.drop([1,2], axis=0, inplace=True)
```

This removes **rows with index 1 and 2**.

df['Name'][0]

```
Aarav Patel
```

df[0:2]

| | Name | Gender | Age | Updated_Marks | Subject |
|---|---|---|---|---|---|
| **0** | Aarav Patel | Male | 25 | 79 | Python |
| **1** | Sophia Smith | Female | 22 | 83 | Python |

df.index

```
RangeIndex(start=0, stop=8, step=1)
```

df.columns

```
Index(['Name', 'Gender', 'Age', 'Updated_Marks', 'Subject'], dtype='object')
```

```
df=df.set_index('Name')
df
```

| | Gender | Age | Updated_Marks | Subject |
|---|---|---|---|---|
| **Name** | | | | |
| **Aarav Patel** | Male | 25 | 79 | Python |
| **Sophia Smith** | Female | 22 | 83 | Python |
| **Liam Johnson** | Male | 25 | 96 | Generative AI |
| **Saanvi Gupta** | Female | 23 | 78 | Statistics |
| **Olivia Brown** | Female | 24 | 86 | Statistics |
| **Indiana Jones** | Male | 21 | 93 | Python |
| **Rohan Sharma** | Male | 25 | 69 | Statistics |
| **James Wilson** | Male | 22 | 81 | Statistics |

```
df.index
```

```
Index(['Aarav Patel', 'Sophia Smith', 'Liam Johnson', 'Saanvi Gupta',
       'Olivia Brown', 'Indiana Jones', 'Rohan Sharma', 'James Wilson'],
      dtype='object', name='Name')
```

```
df.reset_index(inplace=True)
df
```

| | Name | Gender | Age | Updated_Marks | Subject |
|---|---|---|---|---|---|
| **0** | Aarav Patel | Male | 25 | 79 | Python |
| **1** | Indiana Jones | Male | 21 | 93 | Python |
| **2** | James Wilson | Male | 22 | 81 | Statistics |
| **3** | Liam Johnson | Male | 25 | 96 | Generative AI |
| **4** | Olivia Brown | Female | 24 | 86 | Statistics |
| **5** | Rohan Sharma | Male | 25 | 69 | Statistics |
| **6** | Saanvi Gupta | Female | 23 | 78 | Statistics |
| **7** | Sophia Smith | Female | 22 | 83 | Python |

```
df=df.sort_values(by='Updated_Marks')
df
```

| | Name | Gender | Age | Updated_Marks | Subject |
|---|---|---|---|---|---|
| 5 | Rohan Sharma | Male | 25 | 69 | Statistics |
| 6 | Saanvi Gupta | Female | 23 | 78 | Statistics |
| 0 | Aarav Patel | Male | 25 | 79 | Python |
| 2 | James Wilson | Male | 22 | 81 | Statistics |
| 7 | Sophia Smith | Female | 22 | 83 | Python |
| 4 | Olivia Brown | Female | 24 | 86 | Statistics |
| 1 | Indiana Jones | Male | 21 | 93 | Python |
| 3 | Liam Johnson | Male | 25 | 96 | Generative AI |

```
df=df.sort_values(by=['Age','Updated_Marks'])  #if age levels are same the sorting
df                                              happens in Updated_Marks
```

| | Name | Gender | Age | Updated_Marks | Subject |
|---|---|---|---|---|---|
| 1 | Indiana Jones | Male | 21 | 93 | Python |
| 2 | James Wilson | Male | 22 | 81 | Statistics |
| 7 | Sophia Smith | Female | 22 | 83 | Python |
| 6 | Saanvi Gupta | Female | 23 | 78 | Statistics |
| 4 | Olivia Brown | Female | 24 | 86 | Statistics |
| 5 | Rohan Sharma | Male | 25 | 69 | Statistics |
| 0 | Aarav Patel | Male | 25 | 79 | Python |
| 3 | Liam Johnson | Male | 25 | 96 | Generative AI |

```
df=df.sort_index()
df
```

| | Name | Gender | Age | Updated_Marks | Subject |
|---|---|---|---|---|---|
| **0** | Aarav Patel | Male | 25 | 79 | Python |
| **1** | Indiana Jones | Male | 21 | 93 | Python |
| **2** | James Wilson | Male | 22 | 81 | Statistics |
| **3** | Liam Johnson | Male | 25 | 96 | Generative AI |
| **4** | Olivia Brown | Female | 24 | 86 | Statistics |
| **5** | Rohan Sharma | Male | 25 | 69 | Statistics |
| **6** | Saanvi Gupta | Female | 23 | 78 | Statistics |
| **7** | Sophia Smith | Female | 22 | 83 | Python |

**Handling Missing Values**

```
import pandas as pd
import numpy as np
import random

random.seed(101)

names=['Aarav Patel','Sophia Smith','Liam Johnson','Saanvi Gupta','Olivia Brown','Indiana Jones','Rohan Sharma','James Wilson']

genders=['Male','Female','Male','Female','Female','Male','Male','Male']

ages=[random.randint(21,25) for _ in range(8)]
marks=[random.randint(60,95) for _ in range(8)]

subjects=random.choices(['Python','Statistics','Generative AI'],k=8)

df=pd.DataFrame({
    'Name':names,
    'Gender':genders,
    'Age':ages,
    'Marks':marks,
    'Subject':subjects
})

df
```

| | Name | Gender | Age | Marks | Subject |
|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 74 | Python |
| 1 | Sophia Smith | Female | 22 | 78 | Python |
| 2 | Liam Johnson | Male | 25 | 91 | Generative AI |
| 3 | Saanvi Gupta | Female | 23 | 73 | Statistics |
| 4 | Olivia Brown | Female | 24 | 81 | Statistics |
| 5 | Indiana Jones | Male | 21 | 88 | Python |
| 6 | Rohan Sharma | Male | 25 | 64 | Statistics |
| 7 | James Wilson | Male | 22 | 76 | Statistics |

```
df.iloc[4,2]=None
df.iloc[-1,2]=None
df
```

| | Name | Gender | Age | Marks | Subject |
|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25.0 | 74 | Python |
| 1 | Sophia Smith | Female | 22.0 | 78 | Python |
| 2 | Liam Johnson | Male | 25.0 | 91 | Generative AI |
| 3 | Saanvi Gupta | Female | 23.0 | 73 | Statistics |
| 4 | Olivia Brown | Female | NaN | 81 | Statistics |
| 5 | Indiana Jones | Male | 21.0 | 88 | Python |
| 6 | Rohan Sharma | Male | 25.0 | 64 | Statistics |
| 7 | James Wilson | Male | NaN | 76 | Statistics |

```
#the moment we can add None values the datatype of Age from int to change float
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Name     8 non-null      object
 1   Gender   8 non-null      object
 2   Age      6 non-null      float64
 3   Marks    8 non-null      int64
 4   Subject  8 non-null      object
dtypes: float64(1), int64(1), object(3)
memory usage: 452.0+ bytes
```

df.isnull()

| | Name | Gender | Age | Marks | Subject |
|---|---|---|---|---|---|
| **0** | False | False | False | False | False |
| **1** | False | False | False | False | False |
| **2** | False | False | False | False | False |
| **3** | False | False | False | False | False |
| **4** | False | False | True | False | False |
| **5** | False | False | False | False | False |
| **6** | False | False | False | False | False |
| **7** | False | False | True | False | False |

df.isnull().sum()

| | 0 |
|---|---|
| **Name** | 0 |
| **Gender** | 0 |
| **Age** | 2 |
| **Marks** | 0 |
| **Subject** | 0 |

**dtype:** int64

```
df['Age'] =df['Age'].fillna(df['Age'].mean())
df
```

| | Name | Gender | Age | Marks | Subject |
|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25.0 | 74 | Python |
| 1 | Sophia Smith | Female | 22.0 | 78 | Python |
| 2 | Liam Johnson | Male | 25.0 | 91 | Generative AI |
| 3 | Saanvi Gupta | Female | 23.0 | 73 | Statistics |
| 4 | Olivia Brown | Female | 23.5 | 81 | Statistics |
| 5 | Indiana Jones | Male | 21.0 | 88 | Python |
| 6 | Rohan Sharma | Male | 25.0 | 64 | Statistics |
| 7 | James Wilson | Male | 23.5 | 76 | Statistics |

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Name     8 non-null      object
 1   Gender   8 non-null      object
 2   Age      8 non-null      float64
 3   Marks    8 non-null      int64
 4   Subject  8 non-null      object
dtypes: float64(1), int64(1), object(3)
memory usage: 452.0+ bytes
```

```
df['Age']=df['Age'].astype("int")
df
```

| | Name | Gender | Age | Marks | Subject |
|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 74 | Python |
| 1 | Sophia Smith | Female | 22 | 78 | Python |
| 2 | Liam Johnson | Male | 25 | 91 | Generative AI |
| 3 | Saanvi Gupta | Female | 23 | 73 | Statistics |
| 4 | Olivia Brown | Female | 23 | 81 | Statistics |
| 5 | Indiana Jones | Male | 21 | 88 | Python |
| 6 | Rohan Sharma | Male | 25 | 64 | Statistics |
| 7 | James Wilson | Male | 23 | 76 | Statistics |

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   Name     8 non-null      object
 1   Gender   8 non-null      object
 2   Age      8 non-null      int64
 3   Marks    8 non-null      int64
 4   Subject  8 non-null      object
dtypes: int64(2), object(3)
memory usage: 452.0+ bytes
```

```
df1=df.copy()
df1.iloc[4,3]=None
df1.iloc[-1,3]=None
df1
```

| | Name | Gender | Age | Marks | Subject |
|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 74.0 | Python |
| 1 | Sophia Smith | Female | 22 | 78.0 | Python |
| 2 | Liam Johnson | Male | 25 | 91.0 | Generative AI |
| 3 | Saanvi Gupta | Female | 23 | 73.0 | Statistics |
| 4 | Olivia Brown | Female | 23 | NaN | Statistics |
| 5 | Indiana Jones | Male | 21 | 88.0 | Python |
| 6 | Rohan Sharma | Male | 25 | 64.0 | Statistics |
| 7 | James Wilson | Male | 23 | NaN | Statistics |

```
df1=df1.dropna() #drop the rows which have missing values
df1
```

| | Name | Gender | Age | Marks | Subject |
|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 74.0 | Python |
| 1 | Sophia Smith | Female | 22 | 78.0 | Python |
| 2 | Liam Johnson | Male | 25 | 91.0 | Generative AI |
| 3 | Saanvi Gupta | Female | 23 | 73.0 | Statistics |
| 5 | Indiana Jones | Male | 21 | 88.0 | Python |
| 6 | Rohan Sharma | Male | 25 | 64.0 | Statistics |

**Remove Duplicates**

```
#creating a dataframe with duplicates

df2=pd.DataFrame({
    'Name':['Anna','Bob','Bob'],
    'Marks':[88,75,75],
    'Subject':['Python','Java','Java']
})

Df2
```

| | Name | Marks | Subject |
|---|---|---|---|
| 0 | Anna | 88 | Python |
| 1 | Bob | 75 | Java |
| 2 | Bob | 75 | Java |

```
df2.drop_duplicates(inplace=True) #or save to variable df2=df2.drop_duplicates()
df2
```

| | Name | Marks | Subject |
|---|---|---|---|
| 0 | Anna | 88 | Python |
| 1 | Bob | 75 | Java |

#convert the 'Subject' to lowercase

```
df2['Subject']=df2['Subject'].str.lower()
df2
```

| | Name | Marks | Subject |
|---|---|---|---|
| 0 | Anna | 88 | python |
| 1 | Bob | 75 | java |

**Merging Dataframes**

```
df1=pd.DataFrame({
    "Name":['A','B'],
    'Marks':[90,80]
})

df1
```

| | Name | Marks |
|---|---|---|
| 0 | A | 90 |
| 1 | B | 80 |

```
df2=pd.DataFrame({
    "Name":['A','C'],
    'Age':[18,17]
})

df2
```

| | Name | Age |
|---|---|---|
| 0 | A | 18 |
| 1 | C | 17 |

#Merging the dataframe on name

```
pd.merge(df1,df2, on='Name', how='inner')
```

| | Name | Marks | Age |
|---|---|---|---|
| 0 | A | 90 | 18 |

pd.merge(df1,df2, on='Name', how='outer')

| | Name | Marks | Age |
|---|---|---|---|
| 0 | A | 90.0 | 18.0 |
| 1 | B | 80.0 | NaN |
| 2 | C | NaN | 17.0 |

**#Concatenating two dataframes**

```
df3=pd.DataFrame({
    "Name":['D','E'],
    'Marks':[88,92]
})
```

df3

| | Name | Marks |
|---|---|---|
| 0 | D | 88 |
| 1 | E | 92 |

pd.concat([df1,df3])  #axis=0 give vertical concatenation 1 means horizontal  by default axis=0

| | Name | Marks |
|---|---|---|
| 0 | A | 90 |
| 1 | B | 80 |
| 0 | D | 88 |
| 1 | E | 92 |

## Grouping Data with GroupBY

df

| | Name | Gender | Age | Updated_Marks | Subject |
|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 79 | Python |
| 1 | Indiana Jones | Male | 21 | 93 | Python |
| 2 | James Wilson | Male | 22 | 81 | Statistics |
| 3 | Liam Johnson | Male | 25 | 96 | Generative AI |
| 4 | Olivia Brown | Female | 23 | 86 | Statistics |
| 5 | Rohan Sharma | Male | 25 | 69 | Statistics |
| 6 | Saanvi Gupta | Female | 23 | 78 | Statistics |
| 7 | Sophia Smith | Female | 23 | 83 | Python |

#Grouping By "Subject" and fetch the mean 'Marks'

df.groupby('Subject')['Updated_Marks'].mean()

| Subject | Updated_Marks |
|---|---|
| Generative AI | 96.0 |
| Python | 85.0 |
| Statistics | 78.5 |

dtype: float64

df.groupby('Subject')['Updated_Marks'].describe()

| Subject | count | mean | std | min | 25% | 50% | 75% | max |
|---|---|---|---|---|---|---|---|---|
| Generative AI | 1.0 | 96.0 | NaN | 96.0 | 96.00 | 96.0 | 96.00 | 96.0 |
| Python | 3.0 | 85.0 | 7.211103 | 79.0 | 81.00 | 83.0 | 88.00 | 93.0 |
| Statistics | 4.0 | 78.5 | 7.141428 | 69.0 | 75.75 | 79.5 | 82.25 | 86.0 |

## Pivot tables

df.pivot_table(values='Marks',index='Subject', aggfunc=[min,max])

| | min | max |
|---|---|---|
| | Marks | Marks |
| Subject | | |
| Generative AI | 91 | 91 |
| Python | 74 | 88 |
| Statistics | 64 | 81 |

## 1️⃣ groupby

groupby is used to group rows based on a column and then apply an aggregation function (mean, sum, count, etc.).

Your code:

```
grouped = df.groupby('Gender')['Updated_Marks'].mean().reset_index()
grouped
```

**What it does:**

- Groups the dataset by **Gender**
- Calculates **mean of Updated_Marks**
- Returns a simple table

Example OUT:

| Gender | Updated_Marks |
|---|---|
| Female | 82 |
| Male | 78 |

So it answers:

👉 *Average marks per gender*

## 2️⃣ pivot_table

`pivot_table` is used to **reshape the data into a matrix-like table** (similar to Excel Pivot Table).

Your code:

```
pivot=df.pivot_table(values="Updated_Marks",index='Gender',columns="Subject" ,aggfunc='mean')
pivot
```

**What it does:**

- Rows → **Gender**
- Columns → **Subject**
- Values → **Average Updated_Marks**

Example Out:

| Subject Gender | Generative AI | Python | Statistics |
|---|---|---|---|
| Female | NaN | 83.0 | 82.0 |
| Male | 96.0 | 86.0 | 75.0 |

So it answers:
👉 *Average marks of each subject for each gender*

**groupby** → summarize data
**pivot_table** → summarize + reshape

**.apply() and .map()**

```
def assign_grade(marks):
    if marks >= 85:
        return 'A'
    elif 70 <= marks < 85:
        return 'B'
    else:
        return 'C'
```

This function takes a number (marks) and returns a grade:

- >=85 → 'A'
- 70–84 → 'B'
- <70 → 'C'

Using `.apply()` to create a new column

```
df['Grade'] = df['Updated_Marks'].apply(assign_grade)
```

`.apply()` applies a **function to each element of a Series** (or row/column in DataFrame if axis is specified).

Here, `assign_grade` is applied to every value in `df['Updated_Marks']`.

Example:

| | Name | Gender | Age | Updated_Marks | Subject | Grade |
|---|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 79 | Python | B |
| 1 | Indiana Jones | Male | 21 | 93 | Python | A |
| 2 | James Wilson | Male | 22 | 81 | Statistics | B |
| 3 | Liam Johnson | Male | 25 | 96 | Generative AI | A |
| 4 | Olivia Brown | Female | 23 | 86 | Statistics | A |
| 5 | Rohan Sharma | Male | 25 | 69 | Statistics | C |
| 6 | Saanvi Gupta | Female | 23 | 78 | Statistics | B |
| 7 | Sophia Smith | Female | 23 | 83 | Python | B |

So `.apply()` is very **flexible**, because you can pass **any function**, not just a mapping.

**Using .map() for mapping values**

```
df['Gender New']=df['Gender'].map({'Male':0,'Female':1})
df
```

`.map()` is usually used to **map values of a Series according to a dictionary or a function**.

Here, we are converting text `'Male'` and `'Female'` to numerical values `0` and `1`.

Example:

| | Name | Gender | Age | Updated_Marks | Subject | Grade | Gender New |
|---|---|---|---|---|---|---|---|
| 0 | Aarav Patel | Male | 25 | 79 | Python | B | 0 |
| 1 | Indiana Jones | Male | 21 | 93 | Python | A | 0 |
| 2 | James Wilson | Male | 22 | 81 | Statistics | B | 0 |
| 3 | Liam Johnson | Male | 25 | 96 | Generative AI | A | 0 |
| 4 | Olivia Brown | Female | 23 | 86 | Statistics | A | 1 |
| 5 | Rohan Sharma | Male | 25 | 69 | Statistics | C | 0 |
| 6 | Saanvi Gupta | Female | 23 | 78 | Statistics | B | 1 |
| 7 | Sophia Smith | Female | 23 | 83 | Python | B | 1 |

## Handling dates and times in pandas

```
#creating a dataframe wit date strings
df=pd.DataFrame({
    'Name':['A','B','C'],
    'Date':['2024-01-01','2024-02-01','2024-03-01']
})

df
```

| | Name | Date |
|---|---|---|
| 0 | A | 2024-01-01 |
| 1 | B | 2024-02-01 |
| 2 | C | 2024-03-01 |

**df.info()**

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Date    3 non-null      object
dtypes: object(2)
memory usage: 180.0+ bytes
```

**df['Date']=pd.to_datetime(df['Date'])**

**df**

| | Name | Date |
|---|---|---|
| 0 | A | 2024-01-01 |
| 1 | B | 2024-02-01 |
| 2 | C | 2024-03-01 |

**df.info()**

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Date    3 non-null      datetime64[ns]
dtypes: datetime64[ns](1), object(1)
memory usage: 180.0+ bytes
```

# Database

## SQLite (built-in with Python)

SQLite is file-based, so no server setup is required. Python has a **built-in module** `sqlite3`.

## Using `sqlite3`:

```
import sqlite3

# Connect to a database (creates file if doesn't exist)
conn = sqlite3.connect('my_database.db')
cursor = conn.cursor()

# Create table
cursor.execute('''
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    marks REAL
)
''')

# Insert data
cursor.execute("INSERT INTO students (name, marks) VALUES (?, ?)", ("Alice", 95))
conn.commit()

# Query data
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
print(rows)

# Close connection
conn.close()
```

## 'my_database.db'

- This is the database file name.
- If the file does not exist, SQLite will create a new database file with this name in the current directory.

**Using Pandas with SQLite:**

```
import pandas as pd
import sqlite3

conn = sqlite3.connect('my_database.db')

# Read SQL query into DataFrame
df = pd.read_sql_query("SELECT * FROM students", conn)
print(df)

# Write DataFrame to database
df.to_sql('students', conn, if_exists='replace', index=False)
conn.close()
```

**MySQL**

For MySQL, you need:

- MySQL server running
- Python packages: mysql-connector-python or pymysql
- SQLAlchemy for ORM or engine-based connection

Install required packages: pip install sqlalchemy pymysql pandas

**Using SQLAlchemy with MySQL:**

```
from sqlalchemy import create_engine
import pandas as pd

# Format: 'mysql+pymysql://username:password@host:port/database'
engine = create_engine('mysql+pymysql://root:password@localhost:3306/mydb')

# Example: Read table into DataFrame
df = pd.read_sql('SELECT * FROM students', con=engine)
print(df)

# Example: Write DataFrame to MySQL
data = {'name': ['Bob', 'Charlie'], 'marks': [88, 76]}
df_new = pd.DataFrame(data)
df_new.to_sql('students', con=engine, if_exists='append', index=False)
```

**PostgreSQL**

For PostgreSQL, you need:

- PostgreSQL server running
- Python packages: `psycopg2` and `sqlalchemy`

Install required packages: **pip install sqlalchemy psycopg2 pandas**

Using SQLAlchemy with PostgreSQL:

```
from sqlalchemy import create_engine
import pandas as pd

# Format: 'postgresql+psycopg2://username:password@host:port/database'
engine =create_engine('postgresql+psycopg2://postgres:password@localhost:5432/mydb')

# Read table
df = pd.read_sql('SELECT * FROM students', con=engine)
print(df)

# Write DataFrame
data = {'name': ['Dave', 'Eve'], 'marks': [82, 91]}
df_new = pd.DataFrame(data)
df_new.to_sql('students', con=engine, if_exists='append', index=False)
```

## Difference Between SQLAlchemy `.engine` and `pandas.read_sql`

- **SQLAlchemy Engine**: Connects Python to SQL database; can execute raw SQL, ORM operations.

- **Pandas**: `read_sql` / `to_sql` allows reading and writing tables directly using SQLAlchemy engine.

```
from sqlalchemy import create_engine
engine = create_engine('sqlite:///my_database.db')

# Execute raw SQL
with engine.connect() as conn:
    result = conn.execute("SELECT * FROM students")
    for row in result:
        print(row)

# Use pandas
df = pd.read_sql("SELECT * FROM students", engine)
```

# API

An **API is a bridge** that allows two software systems to **communicate** with each other.

- It defines **how requests and responses should happen**.
- Think of it like a **waiter in a restaurant**:
    - You (client) tell the waiter (API) what you want from the kitchen (server).
    - The waiter brings back the food (data/result).

## Why APIs are important

### 1) Integration

- APIs let **different software systems talk to each other**.
- Example:
    - Your app wants weather data → it calls the **OpenWeather API**.
    - Your app doesn't need to know how they calculate weather, just how to ask and receive data.

### 2) Reusability

- Once an API is created, **many apps can use it** without rewriting logic.
- Example:
    - Google Maps API → used in travel apps, food delivery apps, ride-hailing apps.

### 3) Security

- APIs can **control what data is exposed** and what operations are allowed.
- Example: A banking app API gives your account balance but does **not expose the entire database**.

### 4) Scalability

- APIs allow systems to **scale independently**.
- Example: The frontend of a website can fetch data from a backend API. Backend can be upgraded or scaled without touching the frontend.

**5) Automation**

- APIs allow tasks to be automated between systems.
- Example: Automatically updating your CRM when a new customer signs up through your website.

**6) Faster Development**

- Developers can **focus on their core application**, and rely on APIs for external services.
- Example: Payment processing → use **Stripe API** instead of building your own payment gateway.

## Types of API Errors

1. **Client-side errors (4xx)**
    - Examples: 400 Bad Request, 401 Unauthorized, 404 Not Found
    - Cause: Invalid request, missing parameters, wrong API key.
2. **Server-side errors (5xx)**
    - Examples: 500 Internal Server Error, 503 Service Unavailable
    - Cause: Problem in the API server, overload, downtime.
3. **Network errors**
    - Examples: Timeout, DNS failure, connection reset.
    - Cause: Internet issues, firewall, server unreachable.
4. **Data errors**
    - Examples: Unexpected response format, missing fields, empty data.
    - Cause: API changed response structure, bug, or invalid data.

## Best Practices for Error Handling in API Requests

### 1) Always use try-except blocks

- Catch network or request errors.

```
import requests

try:
    response = requests.get("https://api.example.com/data", timeout=5)
    response.raise_for_status()  # Raise exception for 4xx and 5xx
    data = response.json()
except requests.exceptions.HTTPError as http_err:
    print(f"HTTP error occurred: {http_err}")
except requests.exceptions.Timeout:
    print("Request timed out")
except requests.exceptions.ConnectionError:
    print("Connection error")
except Exception as err:
    print(f"Other error occurred: {err}")
else:
    print("Request successful:", data)
```

### 2) Handle HTTP Status Codes

- Don't assume 200 = success. Check status codes explicitly.

```
if response.status_code == 200:
    print("Success!")
elif response.status_code == 404:
    print("Resource not found")
elif response.status_code == 401:
    print("Unauthorized")
else:
    print("Something went wrong:", response.status_code)
```

### 3) Retry Failed Requests

- Temporary network or server issues can be retried.
- Use **exponential backoff** to avoid overloading the server.

```
import time

max_retries = 3
for i in range(max_retries):
    try:
        response = requests.get("https://api.example.com/data", timeout=5)
        response.raise_for_status()
        break
    except requests.exceptions.RequestException:
        wait = 2 ** i  # exponential backoff
        print(f"Retrying in {wait} seconds...")
        time.sleep(wait)
```

### 4) Validate API Response

- Check if JSON has expected fields before using it.

```
data = response.json()
if "results" in data:
    print(data["results"])
else:
    print("Unexpected response structure:", data)
```

### 5) Log Errors

- Always **log errors** for debugging instead of just printing them.

```
import logging

logging.basicConfig(filename="api_errors.log", level=logging.ERROR)
try:
    response = requests.get("https://api.example.com/data")
    response.raise_for_status()
except Exception as e:
    logging.error("API request failed", exc_info=True)
```

### 6) Use Timeouts

- Never wait indefinitely for a response.

```
response = requests.get("https://api.example.com/data", timeout=5)
```

### 7) Handle Rate Limits

- Some APIs limit requests per minute/hour.
- Check **HTTP 429 Too Many Requests** and use `Retry-After` header.

```
if response.status_code == 429:
    wait = int(response.headers.get("Retry-After", 1))
    print(f"Rate limited. Retry after {wait} seconds")
```

### ✅ Summary of Best Practices

1. Use `try-except` for network and HTTP errors.
2. Check HTTP status codes.
3. Retry temporary failures with exponential backoff.
4. Validate response data.
5. Log errors for debugging.
6. Set timeouts.
7. Handle rate limits properly.